Electronics & Computing Club

Coding Dictionary

Build & Code: Arduino Workshop

Ground Rules for Coding

Syntax Rules

- Always end your statements with a semicolon (;) unless shown otherwise (such as after curly braces on if statements)

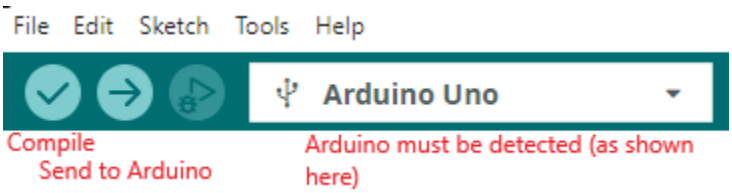
Coding Structure

- Declare variables and objects first outside of setup() and loop()
- Use setup() to execute code only once (to initialize ports and objects)
- Use loop() to execute code in a loop (basically the bulk of your code)
- At the bottom of setup() and loop(), put any custom functions you create or that are given to you

```
// Declare libraries, objects, variables, etc. here
#include <Servo.h>
Servo myServo;
int buttonVal;

void setup() {
  // put your setup code here, to run once
  // Initialize a port
  pinMode(1,INPUT);
  // Initialize an object
  myServo.attach(9);
  // Initialize your serial library
  Serial.begin(9600);
}

void loop() {
  // put your main code here, to run repeatedly
  // Read inputs from your ports
  buttonVal = digitalRead(1);
  // Make decisions
  if(buttonVal == TRUE){

    // Act through your output devices
    myServo.write(90);
    delay(200);
    myServo.write(0);
    // Use your custom functions
    displayMessage();
  }
}

// Write your functions here, at the end
void displayMessage(){
  Serial.println("Button pressed!");
  return;
}
```

Compile and Run

- When you are done coding, click the checkmark icon at the top left to compile your program and detect syntax errors
- When you have no errors from compiling, click the arrow icon at the top left to send your program to the Arduino (requires the Arduino to be plugged in and detected by the IDE)



Terms and Concepts

Comments

- Text that is not part of your code, but can annotate it so you understand it better
- Preceded by // or contained within the characters /* */

example of a comment:

```
// This is a comment
/* This is also a comment,
   but you can make this one on two lines!
*/
```

Variables

- Essentially a "container" for a number, letter or other piece of information
- Always named when you declare it, it's recommended that you make it descriptive enough
- Always associated with a datatype, which determines the exact kind of information you're storing (see datatypes section for more info)
- You can initialize it so it has a starting value

example of variable declaration:

```
int variable1;
float variable2 = 10.2;
bool variable3 = true;
```

Arrays

- A group of variables declared as a list
- Every element of the list is "indexed" (i.e. numbered) starting at 0
- You need to use the index of an element to access it in an array
    - This is done with the array name, followed by the element number in square brackets

example of array declaration:

```
char characterArray[];
int integerArray[] = {1,2,3};
```

example of array manipulation:

```
int result = integerArray[1] + integerArray[2];
// Result equals 5 (2+3=5)
integerArray[0] = 8;
// Changes the first element (indexed as 0) from 1 to 8
```

Strings

- A special case of arrays
- Essentially a character array with a terminating "null" character
- The result is text that you can use in your code (like for sending a message)
- Can be used with a unique syntax for simpler coding (using double quotes)

equivalent declarations:

```
char string1[] = {'H','e','l','l','o','\0'};
char string2[] = {"Hello"};
```

If / If-Else / If-Else-If Statements

- Your main decision statements
- Will decide whether to execute lines of code or not based on a condition statement (true or false)
- There are 3 main flavours of if statements: if, if-else, and if-else-if

example of an if statement:

```
if(variable1 > variable2){
  // Execute code if variable1 is greater than variable2
}
```

example of an if-else statement:

```
if (variable1 > variable2){
  // Execute code if variable1 is greater than variable2
}
else {
```

```
  // Execute code if variable1 is NOT greater than variable2
}
```

example of an if-else-if statement:

```
if (variable1 > variable2){
  // Execute code if variable1 is greater than variable2
}
else if(variable1 == variable2) {
  // Execute code if variable1 is equal to variable2
}
else {
  // Execute code is variable 1 is not greater and not equal to variable2
}
```

Switch/Case Statements

- A fancy kind of if-else-if statement
- Allows you to make decisions based on a single variable
- You can only make decisions based on if a variable is equal to something

example of a switch/case statement:

```
switch(variable1){
  case 1: // Do something if variable1 is equal to 1
  break;
  case 2: // Do something if variable1 is equal to 2
  break;
  default: // Do somehting if variable1 isn't equal to 1 or 2
}
```

While Loops

- Execute something repeatedly until a certain condition has been reached
- Mainly used to execute something until a certain event happens
- May be susceptible to infinite loops if your conditional statement cannot change while your loop is executing

example of a while loop:

```
while(buttonPress == false){
  buttonPress = digitalRead(buttonPin);
  // Continue to loop until the button has been pressed
}
```

For Loops

- Execute something repeatedly for a pre-determined amount of iterations
- Requires three statements within it: a variable declaration and initialization, a statement to remain in the loop, and the action to execute after each loop is done
- The variable declared can be used in your for loop for many useful implementations

example of a for loop:

```
for(int i = 0; i < 10; i++){
/* Execute something 10 times (i starts at 0, and after each loop, increments by
1 and continues to loop until i is no longer less than 10) */
}
```

Functions

- Blocks of code that you can reuse multiple times
- When you use this function, you're basically executing the block of code within it
- Very useful when you have repeating code
- You can decide if your function has inputs an output, neither or both (examples for all 4 cases provided)

Example 1 – no inputs, no output:

```
void function_name() {
  // Reusable code is placed here
  return; // To end the function
}

// To use the function:
function_name();
```

Example 2 – 2 inputs, no output:

```
void function_name(int input1, int input2) {
```

```
  // Reusable code is placed here
  return; // To end the function
}

// To use the function:
function_name(1, variable1);
```

Example 3 – no inputs, one output:

```
int function_name() {
  // Reusable code is placed here
  return function_result; // To end the function and output a number
}

// To use the function:
variable1 = function_name();
```

Example 4 – 2 inputs, one output:

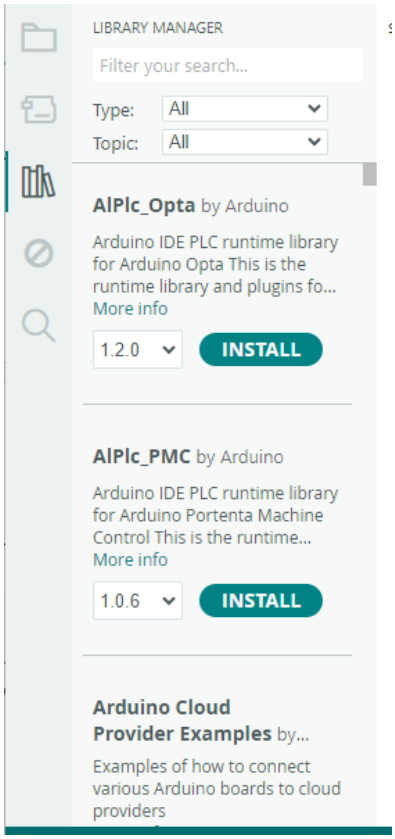
```
int function_name(int input1, int input2) {
  // Reusable code is placed here
  return function_result; // To end the function and output a number
}

// To use the function:
variable1 = function_name(1, 2);
```

Libraries

- Custom made code that you can integrate in your own project code
- Requires that you download it from the library manager

Procedure for downloading a library:

1. Click on the library button in the leftmost menu (the button that looks like a bunch of books) to access the library manager



2. In the "Filter your search" bar, look up the library you need
    - libraries required for our projects will be included in the documentation. Otherwise, you can do research on what libraries you specifically need
    - Sometimes, there are multiple libraries with the same name. You can distinguish between them by looking at the author name
3. When you find your library, click the "Install" button

- Once you have your library downloaded, you need to declare it in your code with an #include statement (this will also be provided in the documentation)

Objects

- A concept in object-oriented programming, where you can associate pins, variables and functions to specific entities
- In our case, an "object" will usually represent a component (i.e. an LCD screen, a sensor, etc.)
- They are declared similarly to variables, except the datatype will be the library name
- All information for specific object declaration will be provided in the project documentation

example of declaring an object:

```
Library_Name object_name([parameters, optional])
```

note: The declaration of an object will differ based on the specific library.

Serial Console

- Uses the Serial library (already present in the Arduino IDE; no #include needed)
- Establishes communication between your computer and your microcontroller
- Useful for receiving information while the code is running

Procedure to use the serial console

1. In your code, initialize communications with Serial.begin() (refer to the Functions section)
2. In your code again, use Serial.print() functions to output messages or variables wherever you see fit
3. Once you upload your code, open the serial console with Ctrl-Shift-M
4. Check the messages, and adjust your code if you see any errors

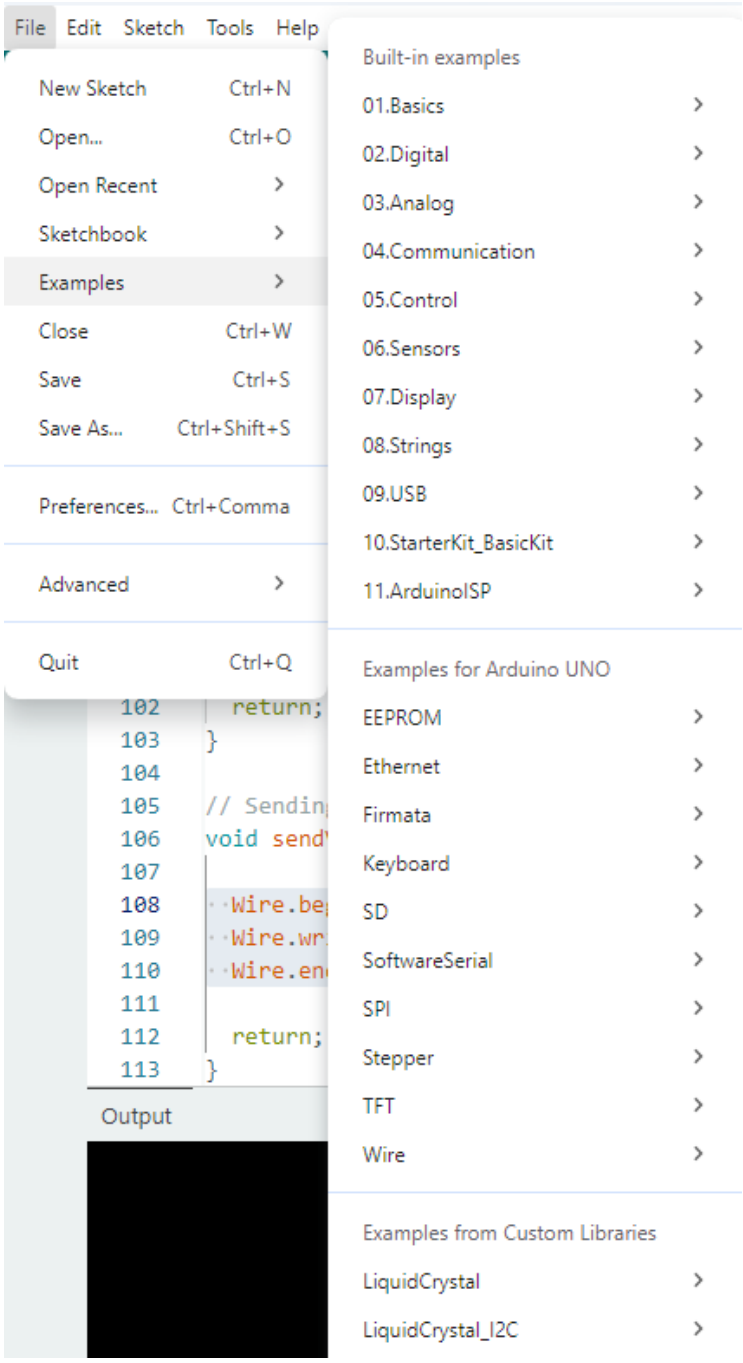
---

Example Files

- Included with every library
- Gives you code for possible use cases of the Arduino
- Libraries provide their own example code so you can learn how to use their functionalities
- Very useful for troubleshooting hardware and exploring functionalities of a library

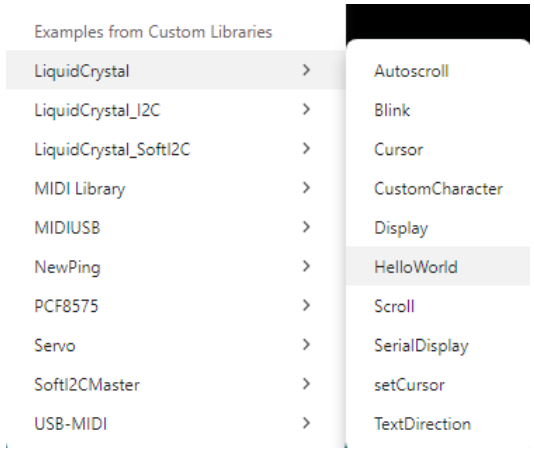
Procedure to access example files:

1. At the top left, go into File > Examples

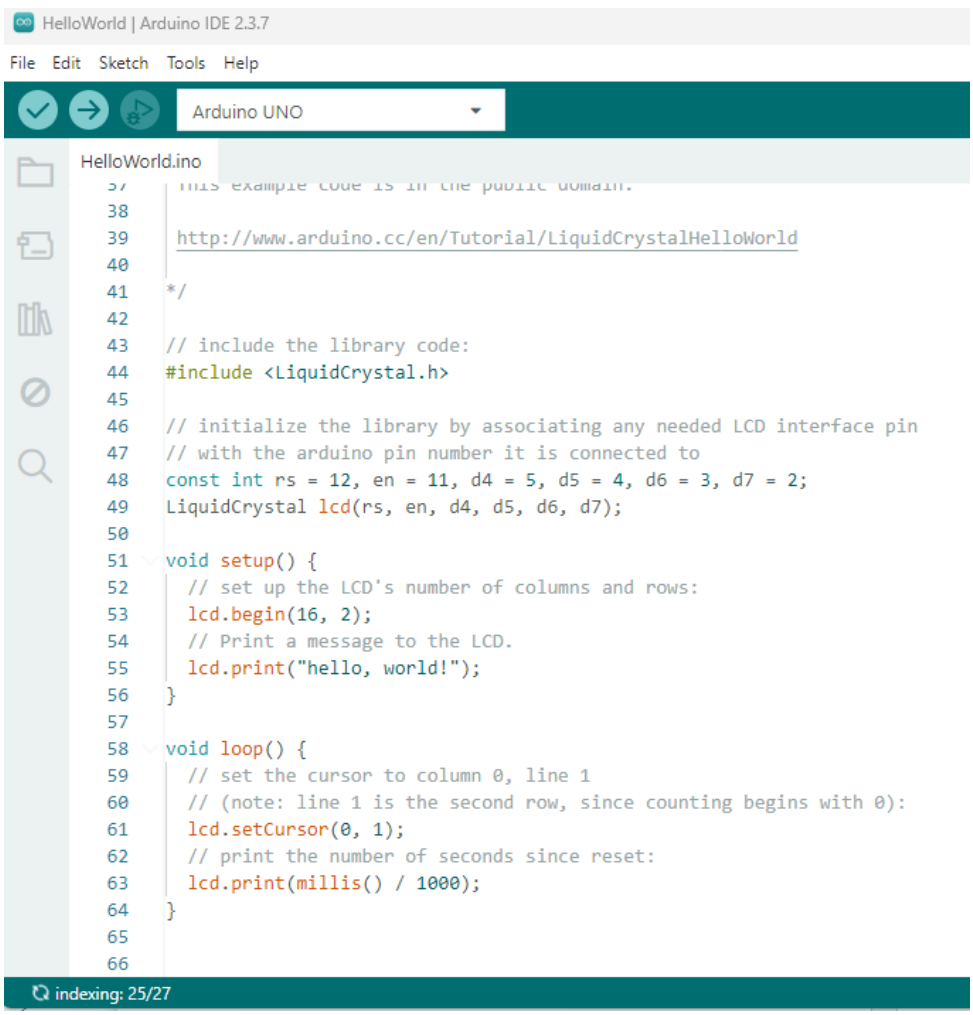


2. You will find the Arduino embedded example files (i.e. not given by a library) at the top, followed by the ones that Arduino Uno already has (i.e. you have to call the libraries for those, but you don't need to download them). Finally, at the end is the custom libraries, which is all the libraries you have downloaded.
3. Open the file you need, study the code, send it to the Arduino if you want, and learn more about libraries!

For example, I have an LCD screen custom library, and I can choose an example file from among the ones offered, like the HelloWorld file.



Once I have clicked on it, I can open the file and see the code! From there, I can upload it to the Arduino and see what happens (if I have the LCD plugged to it), and I can learn from it!



Datatypes

`int`
- Contains all whole numbers (-1,0,1,2,3,4...)
- Can contain numbers from -32,768 to 32,767

`unsigned int`
- Contains all natural numbers (whole numbers without negatives)
- Can contain numbers from 0 to 65,535

`long`
- Contains whole numbers, but with extended capacity
- Can contain numbers from -2,147,483,648 to 2,147,483,647

`unsigned long`
- Same deal as unsigned int
- Can contain numbers from 0 to 4,294,967,295

`char`
- Contains single letters and numbers (encoded in the ASCII format)
- Building blocks of text
- Letters must be contained in single quotes (i.e. 'A' or '6')

`byte`
- Contains one 8-bit value
- Can contain numbers from 0 to 255

`bool`
- Contains a true or false (1 or 0)
- Good for decision making

`float`
- Contains a decimal number (i.e. 0.5, 4.653, 10.2011, etc..)

`const`
- Creates a number that you cannot change (i.e. is not variable)
- Good for giving a name to an Arduino pin (e.g. `const int ledPin = 13;`)

`[]`
- Placed <u>at the end</u> of a variable name
- Indicates that it is an array (multiple variables in a list format)
- example: `int integerArray[] = {1,2,3};`
- More in the "array" entry of the *Terms and Concepts* section

Operators

+, -

- Regular addition and subtraction

++,--

- A special case of addition and subtraction, increments or decrements a number by 1 if you just add it to the end of a variable

e.g.

```
var++;
```

*, /

- Regular multiplication and division
- Look out when dividing! Variables of the `int` datatype can result in `float` outputs if they are divided

=

- Assignment operator
- A variable will receive the result of an operation or just a straight value

e.g.:

```
outputVar = inputVar1 + inputVar2;
// The "=" here gives outputVar the result of adding inputVar1 and inputVar2
```

==

- A logic operator (different from “=”!!)
- Essentially compares two values and asks “are they equal?”

e.g.:

```
inputVar1 == inputVar2
// Is inputVar1 equal to inputVar2? Result is true or false
```

<, >

- Same deal as “==”, asks if a number is less than or greater than
- “<” here means less than, “>” means greater than

<=, >=

- Variants of “<” and “>”
- Means “less than or equal to” and vice-versa

!

- Another logic operator, meaning “not”

e.g.

```
!(variable)
```

```
// If variable is true, result of this is false, and vice-versa
```

Functions

```
void setup()
```

- Runs once at the very beginning of your program
- Use it to initialize things: ports, objects, etc..

```
void loop()
```

- Runs in an infinite loop
- Use it to run the main bulk of your program

```
pinMode([pin #], {INPUT/OUTPUT/INPUT_PULLUP})
```

- Initializes your pins as inputs or outputs
- You can enable a "pullup" feature, which makes the pin a logic 1 by default, and you can turn it into logic 0 by sending a 0 output (they can replace hardware pull-down resistors)

e.g. :

```
pinMode(3, INPUT);
pinMode(4, OUTPUT);
pinMode(5, INPUT_PULLUP);
```

```
digitalWrite([pin #], {LOW/HIGH})
```

- Send a 1 (HIGH) or a 0 (LOW) to one singular pin
- The pin you choose must be initialized as an output

e.g.:

```
digitalWrite(4, HIGH);
digitalWrite(4, LOW);
```

```
digitalRead([pin #])
```

- Read a 1 or a 0 from a pin

- You must assign the result to an int variable
- The pin you choose must be initialized as an input (pullup or otherwise)

e.g. :

```
int stateOfPinThree = digitalRead(3);
int stateOfPinFive = digitalRead(5);
```

```
analogWrite([pin #], [value 0-255])
```

- Sends out an analog value (i.e. not just 1 or 0) to a pin from 0 to 255
- Only pins 3, 5, 6, 9, 10 and 11 can be used for that function (marked with a ~)
- Good for dimming LEDs or motor control, consult a volunteer for possible use cases

e.g. :

```
analogWrite(3, 128);
analogWrite(3, 0);
analogWrite(3, 255);
```

```
analogRead([pin #])
```

- Takes an analog value from a pin, can be from 0 to 1023
- You must assign the result to an int variable
- Only pins A0 – A5 can be used for that function (next to 5V and GND)

e.g. :

```
int valueOfA0Pin = analogRead(A0);
int valueOfA4Pin = analogRead(A4);
```

```
tone([pin #], [frequency of signal], [duration in ms])
```

- Sends a variable-frequency signal to a digital pin at a set amount of time
- Can be sent to a passive buzzer to generate sounds

e.g. :

```
tone(5, 300, 1000); // Generate a 300Hz tone on pin 5 for 1s
```

```
delay([# in ms])
```

- Makes a delay in the program, usually implemented to slow it down
- Very very useful, especially for making readable animations or reliable analog readings

e.g. :

```
delay(200);
```

```
delayMicroseconds([# in micro-seconds])
```

- Same as delay(), but with microseconds

e.g. :

```
delayMicroseconds(2000);
```

```
millis()
```

- Provides the time in milliseconds that the program has been run for
- Very useful in measuring lengths of time

e.g. :

```
unsigned long initialTime = millis();
/* Something happens here */
unsigned long endTime = millis();

unsigned long timeDuration = initialTime - endTime;
```

```
micros()
```

- Same deal as millis(), but with microseconds

```
map([variable],[min of input range], [max of input range], [min of output range],
[max of output range])
```

- Takes a value and changes it based on a possible input range and a desired output range
- Useful for mapping values between ranges (e.g. between analogRead() and analogWrite())

e.g. :

```
int value = analogRead(A3); // analogRead() ranges from 0 to 1023
map(value, 0, 1023, 0, 255);
analogWrite(3,value); // value variable is now ranging from 0 to 255
```

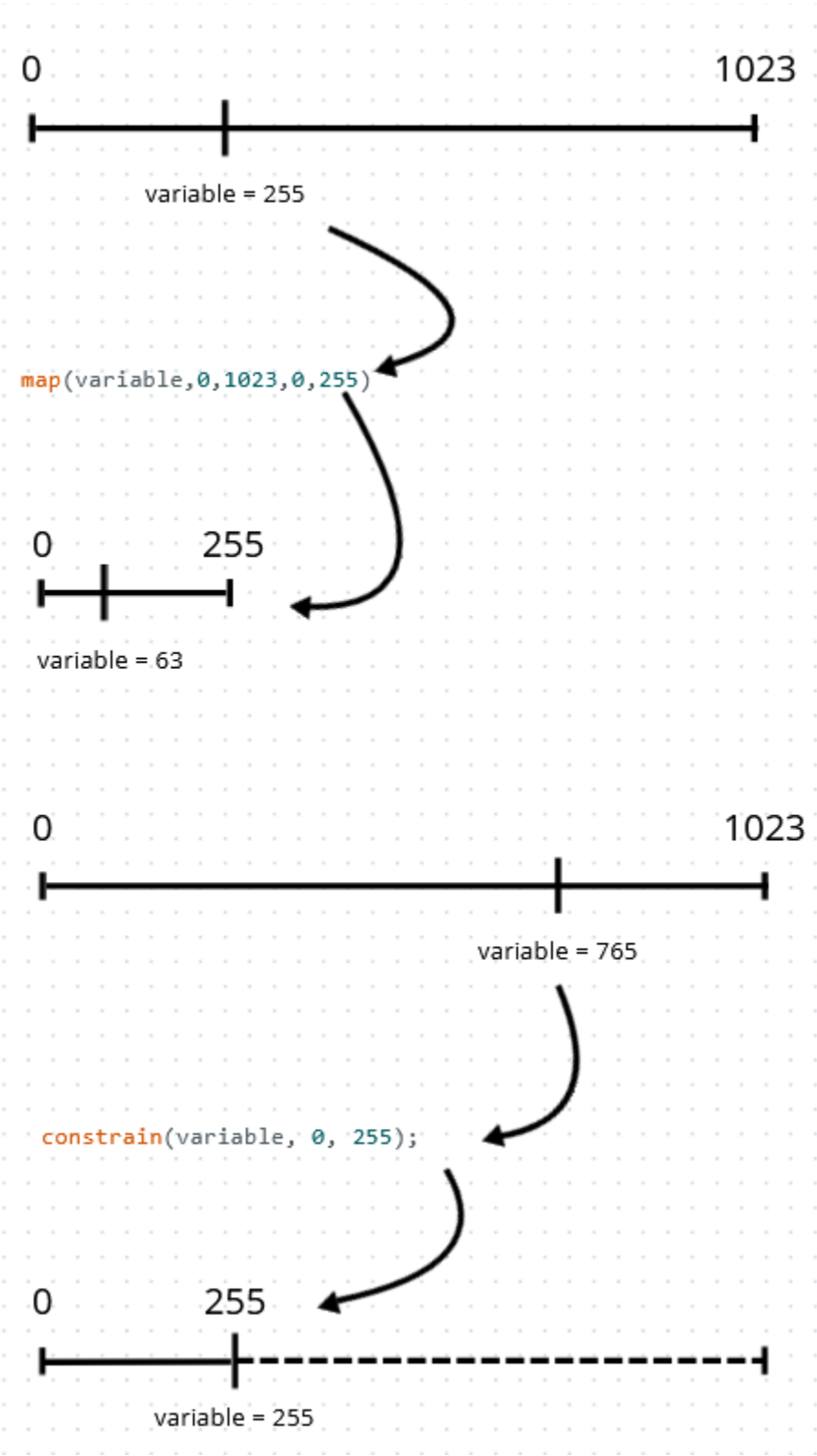
```
constrain([variable],[min of range], [max of range])
```

- Takes a value and truncates it to fit within a range of values
- If your value is outside that range, it will be sent to the nearest value it can have (i.e. the minimum or maximum value)

e.g. :

```
int value = 765;
constrain(value, 0, 255);
Serial.println(value); // Value will be 255
```

Difference between map() and constrain(), illustrated:



```
pulseIn([pin #], {HIGH/LOW})
```

- Get a length of time in microseconds for how long a pin is high or low for

e.g. :

```
duration_of_pulse = pulseIn(12, HIGH);
```

```
Serial.begin([baud rate])
```

- Initialize serial communications (i.e. the serial console)
- Necessary before using any Serial.print function
- Baud rate needs to be specified here (you can set it to 9600, as this is the default for the serial console)

e.g.:

```
Serial.begin(9600);
```

```
Serial.print([text or variables])
```

- The main way to send messages to the serial console
- In the brackets, add strings or a variable you want to display
- You can use its sister function, Serial.println(), to create a new line (basically it appends the text with an enter key)

e.g.:

```
Serial.print("This message will precede variable1 : ");
Serial.println(variable1);
```

Useful Tools

Lookup Tables

- A major solution to "decoding" in a program
- Can be thought of as a literal table, with an input and output pair, where you can choose a certain input to get a certain output.

I send number 2 ->

Input value	Output value
1	10
2	20
3	30

-> I receive number 20

This lookup table is equivalent to multiplying the input by 10, so it's not super useful.

However, imagine if we needed a "2" to be decoded into a "40". You can't just multiply the number by 10, so instead a lookup table will just tell you directly to pick 40. In that way, a lookup table is more flexible than a math function, even though they're both doing the same thing.

I send number 2 ->

Input value	Output value
1	10
2	40
3	30

-> I receive number 40

So how do we do this in a program? Well, we can create two lists and simply associate each index on the list with a certain "row" of our table.

```
int inputNumbers[3] = {1,2,3};
int outputNumbers[3] = {10,40,30};
```

We can use this as a lookup table because the index (the number inside the square brackets) can be used to refer to two numbers at the same time (e.g. index "1" here can be equal to 2 for inputNumbers[] and 40 for outputNumbers[]).

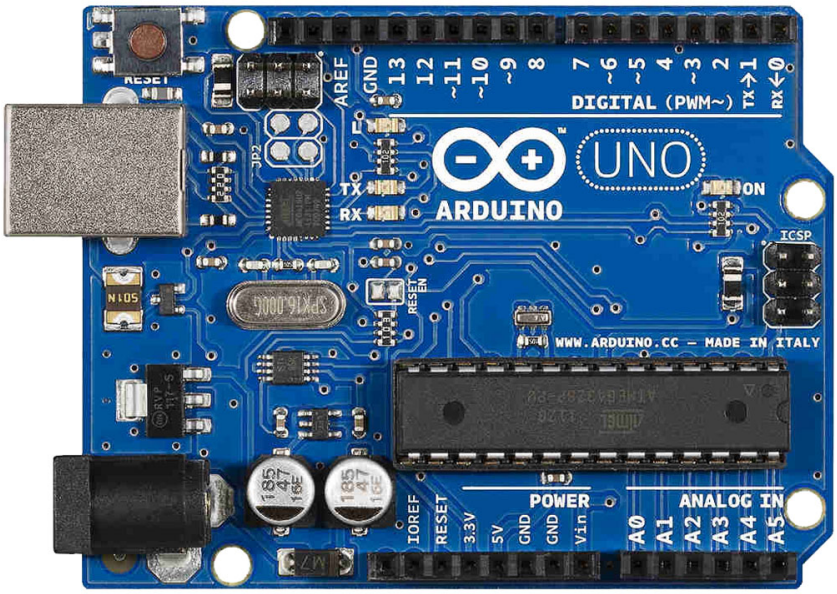
You can implement this in a for loop to get a true lookup table. In the example below, you are comparing an input variable with every element of the inputNumbers[] list. Once you find a match, you assign the output variable to a corresponding element of the outputNumbers[] list. See how "i" is in the bracket for both lists? In this way, both lists are linked, and the output value will be whatever the input value was paired with in your lookup table.

```
for(int i = 0; i < 3; i++){
  if(input == inputNumbers[i]){
    output = outputNumbers[i];
  }
}
```

Hardware Components

Arduino Uno R3

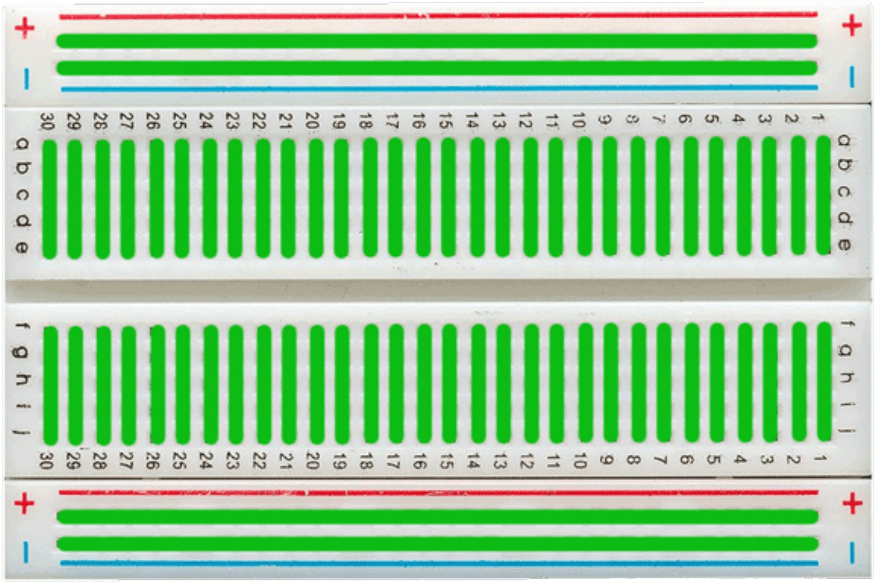
- The brains of your project, will perform all tasks that are written in code
- You program and power it by plugging it through the USB cable and using the Arduino IDE
- Both rows have each pin labelled, which you can refer to in code (whenever a function says "pin #")
- All pins labelled with "~" have PWM capabilities
- All pins starting with A (i.e. A0) are analog pins (they can read more than just 1 or 0)



Breadboard

- Platform on which circuits are made
- Contains holes that are interconnected in a certain pattern
- Allows you to easily make circuits by plugging components in the holes
- Red and blue lines are usually for power (5V, GND), while middle rows are for regular components

Reference: all green lines are interconnected

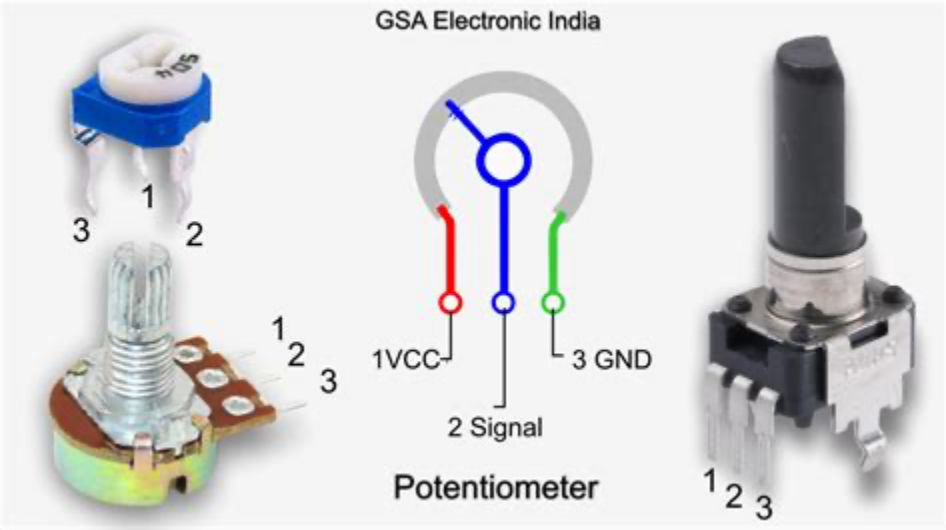


Resistor

- Standard component with many, many different uses
- Unit of measurement is the Ohm (Ω)
- Limits the flow of current according to Ohm's Law (Voltage(V) = Current (A) x Resistance(Ω))
- Functions: pull-down logic for buttons, current limiting for LEDs, etc.

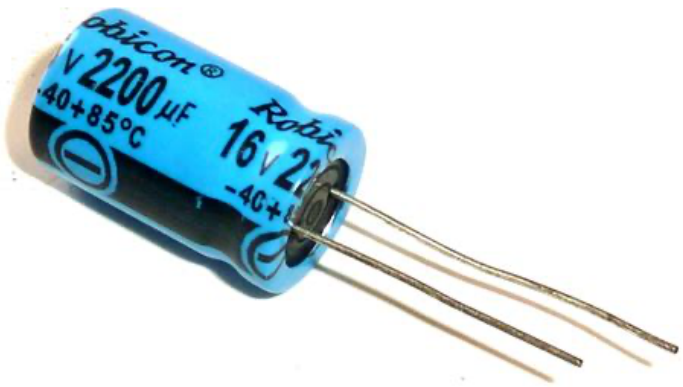
Potentiometer

- A more exciting resistor
- Includes a knob to vary the resistance value
- Very useful if you want to send a value between 0V and 5V to an analog pin



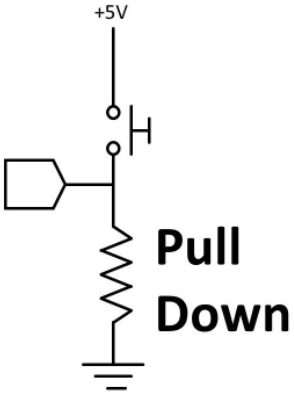
Capacitor

- A standard component like the resistor
- Unit of measurement is the Farad (F), you mainly find them in the micro-Farad (μF), which is 0.000001F
- Used to store and release energy, can “smooth out” a voltage so it’s consistent
- Functions: power smoothing (making a power line less prone to voltage dips and spikes during activity)
- IMPORTANT: if your capacitor looks like the picture here, make sure to connect the short lead (black ribbon side) to ground and the long lead to positive voltage. Connecting in reverse from that can damage the component!



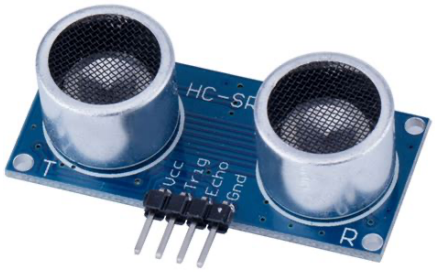
Button

- A switch that you can press
- Good for creating inputs and interacting with your code
- Requires a pull-down resistor to function properly

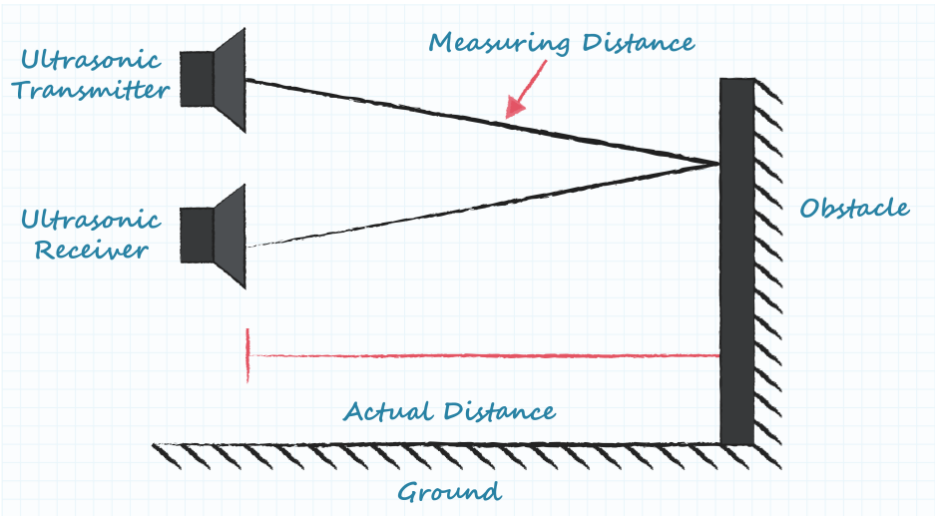


Proximity Sensor (HC-SR04)

- Detects distance from an obstacle using ultrasounds
- Library used: *NewPing* by Tim Eckel

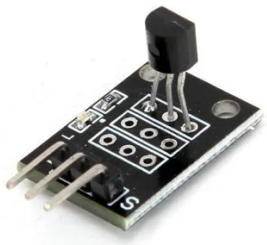


-



18B20 Temperature Sensor

- A versatile digital thermometer that detects temperature at an accuracy of +/-0.5°C
- Libraries used: *OneWire* by Jim Studt, Tom Pollard, et al. and *DallasTemperature* by Miles Burton, Tim Newsome, et al.
- **G** pin connects to ground, **R** pin goes to +5V, and **Y** pin is the output signal that goes to the Arduino



CD4511 BCD-to-7-Segment Decoder

- Converts an incoming 4-bit binary stream to seven signals that controls an external display
    - It allows for binary numbers to be displayed as decimal numbers on an LED display

- The **Vdd** pin connects to +5V, and the **GND** pin connects to ground
- Pins **D0** through **D3** are for the input binary signal
- For proper operation, the **LT** and **BL** pins must be pulled HIGH (tied to +5V), and the **LE** must be pulled LOW (tied to ground)
- Pins **a** through **f** correspond to the output signals that connect, through external resistors, to a 7-segment LED display.

Passive Buzzer

- A small piezo-electric component that converts electrical signals to sound waves
    - When a square wave (an alternating HIGH/LOW signal) is passed to the passive buzzer, an internal element vibrates and produces sound
    - Changing the frequency of the incoming signal changes pitch, while changing the duty cycle affects volume
    - You wire one side to a digital PWM signal, and the other to ground.

Active Buzzer

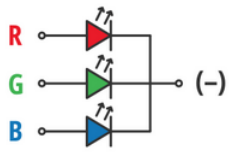
- The active buzzer is the same as the passive buzzer, although it requires an additional pin to be connected to an external source of power (5V)
- This particular module uses the "**-**" pin for ground, the middle pin for power (5V), and the **S** pin for the incoming PWM signal.
- **DO NOT REMOVE THE STICKER** – without it, the active buzzer can become extremely loud!

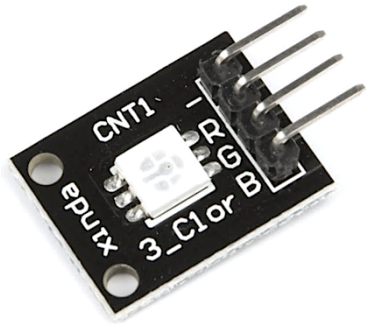
LED (Light-Emitting Diode)

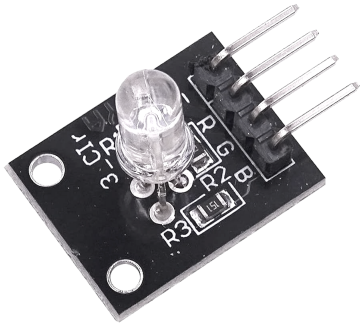
- An essential component in electronics that converts electricity to light
- The shorter leg of the component (the **cathode**) is connected to ground, and the longer leg (**anode**) connects to power (in our case, 5V)
- Typically, a resistor is added in series with the LED to limit the current it consumes. Otherwise, it can overheat or burn.

RGB LED (Red, Green, Blue Light-Emitting Diode)

- While it may resemble a typical LED on the surface, this one actually has three integrated LEDs, allowing you to control the emitted red, green, and blue light intensities.
- The board that the LED comes on includes current-limiting resistors, so no external components are needed to use this in circuits.
- Both images below are RGB LEDs, despite their visual differences. Their pinouts are also the same, where you would supply a PWM signal from the Arduino to each of its **R**, **G**, and **B** pins, and connect the "**-**" pin to ground.

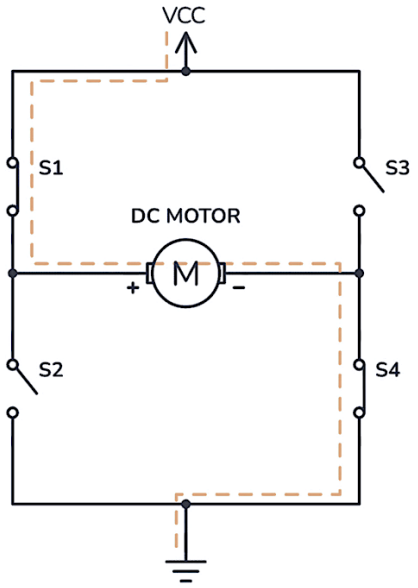






DC Motor

- A DC motor converts electrical energy into mechanical energy
    - Magnets embedded within the motor create a constant magnetic field, and when current flows through the coil inside, the interaction between the coil's magnetic field and the magnets produces a force that causes the rotor to spin
- Since this is considered to be an inductive load (because a coil is an inductor), a reverse-biased flyback diode placed across the motor's terminals is best practice to avoid current spikes.

H-Bridge Configuration

- It is a 4 switch electronic circuit that enabled DC motors to run forward, backward, brake, or coast by reversing voltage polarity
    - The image below shows a simplified version of this configuration using switches instead of transistors or MOSFETs.
- Certain integrated circuits (ICs) come pre-packaged with half H-bridge configurations, like the L239D. Since it is a half-H bridge configuration, two ICs are needed to control 4 motors. These ICs also include protection diodes, so no external components are needed in that regard.